DAY 11 OF 30

Notebooks and Spark

Where the heavy processing happens.
And how Fabric changes the experience.

PySpark | Starter pools | Environments

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE PROBLEM

One machine runs out sooner than you think.

1	**A few million rows**	A laptop is fine
2	**A few billion**	It runs for hours, or crashes
3	**The usual fix**	Buy a bigger machine

There is always a bigger dataset. **A bigger machine is not a strategy.**

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE SIMPLE VERSION

Many cooks, one big order.

One cook cannot serve a banquet alone. A head chef splits the order, hands parts to many cooks, and plates the result.

Spark is the **head chef**. It splits your job across many machines and combines the answers.

✓ **Your code**	Written once, in a notebook
✓ **Spark**	Splits the work and runs it
✓ **You**	Never manage the machines

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

HOW IT WORKS

Running Spark in Fabric

1. Open a **notebook** and attach a Lakehouse
2. A **starter pool** gives you a session in seconds
3. Write **PySpark, Spark SQL, Scala or R**
4. Results save as **Delta tables** in OneLake
5. **Schedule** it, or call it from a pipeline

No cluster to build

Fabric keeps pools ready. You pick a size only when the default is not enough.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

FOR ENGINEERS

Settings that matter

Starter pool or custom pool

Starter pools start fast with default sizing. Custom pools let you choose node size and autoscale limits.

Environments hold your setup

Libraries and Spark settings live in an environment, so every notebook and job runs the same way.

High concurrency mode

Several notebooks can share one session, which saves start-up time and capacity.

Native execution engine

A vectorised engine you switch on in the environment. It can speed up queries with no code change.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE HONEST PART

Do you need Spark?

YES WHEN

- Data is large or growing fast
- Logic is complex and needs testing
- Files are raw or semi-structured

MAYBE NOT WHEN

- The data fits comfortably in SQL
- The team does not know Python
- A plain Python notebook can handle it

> **The catch:** Spark is easy to start and easy to waste. A notebook left running, or a job on a pool far larger than it needs, quietly eats the shared capacity.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

FROM REAL WORK

Spark on real projects

PySpark has been the core of most platforms I have worked on.

1. **Bronze, Silver and Gold** layers for a people-analytics platform
2. **A sustainability reporting pipeline** that cut manual effort by 80%
3. **Daily volumes** from hundreds of GB to several TB on a Databricks platform
4. **SQL stored procedures** rewritten as PySpark during a migration

Same skills, whether the engine runs in **Fabric or Databricks**.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

SAVE THIS

Spark in Fabric in five lines

1. **Spark** splits big jobs across machines
2. **Notebooks** are where you write the code
3. **Starter pools** remove the waiting
4. **Environments** keep runs consistent
5. **Idle sessions** cost capacity, so stop them

Tomorrow: Incremental loads.
Moving only what changed.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Follow for Day 12